

Embedded System Design Guide

Sensor Monitoring System

Version 1.2 | 2025-03

This document describes the architecture, design decisions, and implementation guidelines for the sensor monitoring subsystem used in embedded control units. It covers hardware abstraction, interrupt handling, calibration algorithms, and alert management.

Target audience: Firmware engineers with C/RTOS experience.

Chapter 1: System Architecture

1.1 Overview

The sensor monitoring system consists of three main components: the hardware abstraction layer (HAL), the data processing pipeline, and the alert management module.

The HAL isolates hardware-specific code (register access, interrupt setup) from the application logic. This separation allows unit testing without physical hardware and simplifies board bring-up.

1.2 Component Diagram

The following table summarizes the primary modules and their responsibilities:

Module	File	Responsibility
Sensor HAL	sensor.c	Read raw ADC values, manage IRQ buffer
Logger	logger.c	Leveled log output to stderr/file
Main Loop	main.c	Initialization, polling, shutdown
Calibration	sensor.c	Convert raw values to physical units
Alert Manager	sensor.c	Threshold comparison, level assignment

1.3 Data Flow

- Hardware IRQ fires on new ADC sample.
- sensor_irq_handler() pushes raw byte into circular buffer (g_data_buf).
- Main loop calls process_all_sensors() at POLL_INTERVAL_MS cadence.
- sensor_read() pops from the buffer and applies calibration.
- check_alert() compares calibrated value against per-sensor thresholds.
- If alert fires, log_alert() formats and emits the log entry.

Chapter 2: Interrupt Handling and Concurrency

2.1 Circular Buffer

The IRQ handler and main loop share a circular byte buffer. The buffer head pointer (`g_buf_head`) is written only by the IRQ handler; the tail pointer (`g_buf_tail`) is read/written only by the main loop.

IMPORTANT: On single-core MCUs without preemption between ISR and main, this one-producer/one-consumer pattern is safe without a mutex. However, on multi-core or preemptible RTOS environments, atomic operations or a spinlock MUST be used.

2.2 Volatile Qualifier

All shared variables are declared volatile to prevent the compiler from caching them in registers:

- `g_data_buf[DATA_BUF_SIZE]`
- `g_buf_head`, `g_buf_tail`
- `g_irq_fired`

Without volatile, optimizing compilers may hoist reads out of loops, causing the main loop to never observe IRQ updates.

2.3 Known Limitations

The current implementation has the following known limitations:

- Buffer overflow is silently dropped (new sample discarded when full).
- No timestamp is attached to samples (RTC not yet integrated).
- Only single-byte samples are supported; multi-byte sensors need framing.
- The IRQ handler does not debounce or validate the ADC reading.

Chapter 3: Calibration Algorithm

3.1 Linear Calibration Model

Each sensor uses a linear calibration model:

$$\text{calibrated} = \text{raw} * \text{scale} + \text{offset}$$

The coefficients are stored in static arrays `s_calib_scale[]` and `s_calib_offset[]`, indexed by sensor ID. They are set during `sensor_init()` based on sensor type.

3.2 Per-Sensor Coefficients

Sensor ID	Type	Scale	Offset	Unit
0 (TEMP)	Temperature	0.01	-40.0	degC
1 (HUMID)	Humidity	0.1	0.0	% RH
2 (PRESS)	Pressure	0.01	800.0	hPa

3.3 Calibration Update Procedure

To update calibration coefficients in the field:

1. Place sensor in a known reference environment.
2. Read 100 samples and compute the mean raw value.
3. Solve for offset: $\text{offset} = \text{reference} - \text{mean_raw} * \text{scale}$.
4. Write new offset to non-volatile storage.
5. Call `sensor_init()` to reload coefficients on next boot.

Note: scale coefficient is assumed stable and not field-adjustable in the current firmware version.

Chapter 4: Alert Management

4.1 Alert Levels

The system defines three alert levels:

Level	Value	Meaning	Log Destination
ALERT_NONE	0	No issue detected	-
ALERT_WARNING	1	Threshold exceeded	LOG_WARN
ALERT_CRITICAL	2	Threshold x1.5 exceeded	LOG_ERROR

4.2 Temperature Alert Thresholds

Temperature alerts use a symmetric threshold relative to a configured limit T:

CRITICAL : calibrated > T * 1.5 OR calibrated < -T * 1.5

WARNING : calibrated > T OR calibrated < -T

Default threshold T for temperature sensor: 85.0 degC

4.3 Humidity and Pressure Alerts

Humidity:

WARNING : calibrated > 90.0 %RH

CRITICAL : calibrated > 95.0 %RH (note: implementation bug - see errata)

Pressure:

WARNING : calibrated < 900.0 hPa OR calibrated > 1100.0 hPa

4.4 Errata: Humidity CRITICAL Unreachable

BUG (tracked as ISSUE-42): In `check_alert()`, the `ALERT_CRITICAL` branch for humidity (>95.0) is unreachable because the preceding `WARNING` branch (>90.0) returns first. The condition ordering must be reversed:

Correct order:

if (calibrated > 95.0) return `ALERT_CRITICAL`;

if (calibrated > 90.0) return `ALERT_WARNING`;

Chapter 5: Coding Guidelines and Known Issues

5.1 Buffer Safety

All string formatting operations MUST use `sprintf()` with explicit size limits. The current `log_alert()` implementation uses `sprintf()` without bounds checking (BUG: ISSUE-43). This must be fixed before production release:

```
/* WRONG - no bounds check */
sprintf(msg, "[ALERT] level=%d sensor=%d ...", ...);

/* CORRECT */
snprintf(msg, sizeof(msg), "[ALERT] level=%d sensor=%d ...", ...);
```

5.2 Uninitialized Variable Warning

`sensor_read()` declares `'int16_t raw;'` without initialization. On builds where the hardware register read macro is not defined, `raw` retains an indeterminate value. Always initialize local variables at declaration (C99 allows this):

```
int16_t raw = 0; /* safe default until hardware read succeeds */
```

5.3 Unused Code

`_sensor_selftest()` is defined but never called. Dead code increases binary size and confuses code reviewers. Either integrate it into the CI test suite or remove it.

5.4 Build Requirements

Item	Requirement
Compiler	GCC 10+ or Clang 12+
C Standard	C99 minimum, C11 preferred
Warnings	-Wall -Wextra -Werror recommended
Optimization	-O2 for release, -O0 -g for debug
RTOS	FreeRTOS 10.x (optional, single-threaded mode supported)